

Sim-to-Real Aware End-to-End Learning Environment for Micromobility

Shouma Amano¹ and Takuya Azumi²

Abstract—While end-to-end autonomous driving systems show promise, their application to micromobility vehicles is hindered by simulators failing to capture specific kinematics, such as differential drives and omni-wheels. This paper proposes a sim-to-real-aware, vehicle-specific end-to-end learning environment for the WHILL Model CR on AWSIM and ROS 2. To minimize the sim-to-real gap, physical parameters are optimized via Bayesian optimization using real-world data, reducing trajectory errors across various driving scenarios. Additionally, this study introduces a synchronized architecture tailored for the stable training of world model-based agents. An end-to-end policy trained with DreamerV3 exhibited learning progress and achieved task completion in a simulated obstacle avoidance setting. Furthermore, this policy demonstrated direct sim-to-real transfer to the physical vehicle, enabling the vehicle to navigate around a cardboard box in a real-world corridor replica without fine-tuning. This paper provides a practical foundation for sim-to-real micromobility policy studies.

Index Terms—End-to-End, Autonomous Driving Simulator, Micromobility, Sim-to-Real

I. INTRODUCTION

Autonomous driving systems [1], [2] are expanding to micromobility vehicles, such as delivery robots and electric wheelchairs. This technology enables inclusive, efficient, and sustainable urban environments through novel transportation and last-mile logistics. However, integrating these vehicles requires systems capable of complex interactions with pedestrians and dynamic obstacles. Traditional modular systems face challenges in these environments, particularly suboptimal module coordination and error propagation from perception to planning. End-to-end (E2E) [3], [4], [5] approaches address these challenges by utilizing a single, unified network mapping raw sensor data to driving commands. Furthermore, the growing use of world models [6], [7], [8], [9] enables agents to build an internal representation of their environment and simulate future outcomes, allowing for proactive, long-term planning.

While these learning-based methods have largely focused on conventional automobiles, applying them to micromobility vehicles presents two barriers. First, real-world testing on unpredictable sidewalks is costly and risky, making simulation an essential tool. Second, current mainstream autonomous driving simulators are insufficient, as they are primarily designed to replicate the dynamics and traffic flow of cars. Standard simulators often fail to capture the complex kinematics inherent to micromobility vehicles, such as differential drive systems and omni-wheel behaviors. This

physical discrepancy creates a sim-to-real gap, which hinders the deployment of E2E driving policies.

To overcome these barriers, this study proposes an E2E learning environment tailored for the WHILL Model CR using AWSIM [10] and ROS 2. To minimize the sim-to-real gap, a system identification framework utilizing Bayesian optimization is implemented to tune physical parameters based on real-world driving data. Furthermore, to support E2E learning, this study introduces a layered architecture featuring a hybrid synchronization protocol. This protocol bridges the asynchronous ROS 2 data streaming with episodic reinforcement learning requirements, ensuring stable training for world model-based agents. This E2E learning environment enables the direct sim-to-real transfer of the learned policy to the physical vehicle without additional real-world training.

The main contributions of this paper are as follows:

- Minimizing the sim-to-real gap in trajectory and yaw angle responses by optimizing physical parameters based on real-world driving data.
- Establishing an E2E learning infrastructure for micromobility vehicles using ROS 2 and AWSIM, demonstrated to support E2E driving policy training.
- Demonstrating direct sim-to-real transfer of an E2E driving policy in the reported corridor scenario, enabling collision-free obstacle avoidance without updating the network weights.

Based on these contributions, this study addresses the following research questions:

- RQ1: To what extent can the sim-to-real gap in trajectory and yaw angle be minimized by optimizing physical parameters based on real-world driving data?
- RQ2: In the proposed environment, can a world model-based E2E policy (DreamerV3) trained solely on RGB images achieve stable obstacle avoidance performance, as measured by reward curve?
- RQ3: Can the vision-based E2E policy, trained in the proposed environment, achieve collision-free obstacle avoidance on a physical vehicle without network weight fine-tuning?

The paper is organized as follows: Section II describes the system model, and Section III details the proposed environment. Section IV reports the evaluation, Section V discusses related work, and Section VI concludes.

II. SYSTEM MODEL

The proposed system model integrates a simulator, a micromobility vehicle, and a world model-based driving

¹Saitama University amano.s.388@ms.saitama-u.ac.jp

²Academic Association (Graduate School of Science and Engineering) Saitama University

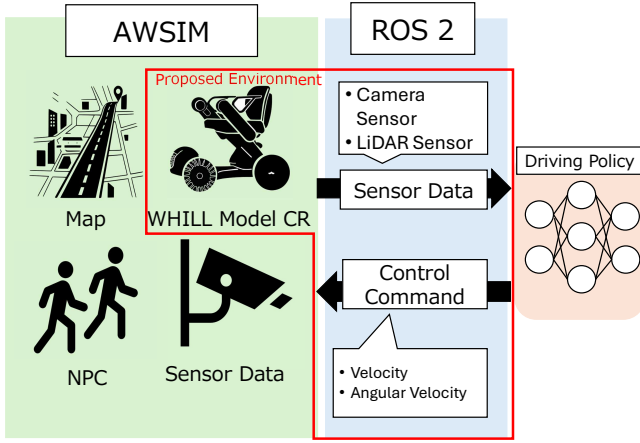


Fig. 1: System Model.

policy via middleware, as depicted in Fig. 1.

A. AWSIM

AWSIM is a digital twin-oriented autonomous driving simulator designed to replicate real-world environments. In this study, a detailed map of Nishi-Shinjuku, Tokyo, is utilized and populated with Non-Player Characters (NPCs) to collect the multimodal sensor data (camera and LiDAR) required for training the agent.

B. WHILL Model CR

The target vehicle is the WHILL Model CR, a micromobility vehicle characterized by a differential drive system for propulsion and specialized front omni-wheels. This combination enables distinct kinematic behaviors, such as zero-radius turns with minimal lateral friction, which influence the range of actions the driving policy must learn.

C. ROS 2

ROS 2 [11] bridges the AWSIM simulator and the learning agent. This middleware handles the asynchronous transmission of sensor data to the agent and relays velocity control commands back to the vehicle.

D. Driving Policy

The driving policy employs DreamerV3 [12], a world model-based reinforcement learning algorithm. DreamerV3 was selected because the reported task requires learning continuous driving commands from low-resolution RGB observations under a limited number of simulator interactions, where model-based latent dynamics can improve sample efficiency and temporal prediction compared with purely reactive policies. This choice is also consistent with prior physical robot learning studies using world models [9]. Within the proposed environment, this E2E policy functions as a neural controller that maps raw sensor observations directly to driving commands, adapting to the specific kinematics of the micromobility vehicle.

TABLE I: Definitions of coordinate systems and reference frames

Category	Symbol	Definition & Description
Coordinate System	Σ_{base}	Right-Handed (ROS Std.) Defined at the Model Origin. +x: Forward, +y: Left, +z: Up.
CoM Offsets	$CoM_{x,y,z}$	Relative to Σ_{base} Offsets from the Model Origin. CoM_x : Front, CoM_y : Left, CoM_z : Up.
Control Inputs	v, ω	Ref. at Rear Axle Center v : Linear velocity (+x). ω : Angular velocity (yaw).
Evaluation	Σ_{eval}	Odometry Frame Tracks the 2D pose of the Rear Axle Center relative to the initial starting position.

III. DESIGN & IMPLEMENTATION

The proposed integrated E2E learning environment architecture utilizes AWSIM and ROS 2 to enable world model-based reinforcement learning for micromobility vehicles. The WHILL Model CR is modeled within this environment to provide a foundation for training and deploying E2E driving policies, as illustrated in Fig. 2.

A. E2E Learning Environment and Control Interface

A 3D model of the WHILL Model CR is integrated into the AWSIM environment. The simulation utilizes a map of Nishi-Shinjuku to collect multimodal sensor data in wheelchair-accessible areas. To bridge the internal physics engine with the ROS 2 ecosystem, the coordinate systems and kinematic reference points are defined, as illustrated in Fig. 3 and Table I. The vehicle is controlled via standard ROS 2 `geometry_msgs/Twist` messages published to the `cmd_vel` topic, defining target linear (v) and angular (ω) velocities. Furthermore, the proposed environment supports the evaluation of direct sim-to-real transfer without network weight fine-tuning. For this study, a virtual replica of a university laboratory corridor was constructed. This scenario incorporates 3D models of task-specific objects, such as a cardboard box and a goal marker, to evaluate visual obstacle avoidance.

B. Vehicle Modeling and Parameter Optimization

To bridge logical velocity commands and the physics engine, a force-based control algorithm designed for the differential drive system of the WHILL Model CR is utilized. Target linear (v_{tgt}) and angular (ω_{tgt}) velocities are decomposed into left and right wheel velocities using standard inverse kinematics. Independent PID controllers then calculate the required acceleration for each wheel to minimize velocity tracking errors. Finally, these accelerations are mapped to actual tractive forces (F_{left}, F_{right}) applied to the simulated wheels, assuming a simplified mass distribution.

A system identification problem is formulated to minimize the sim-to-real gap. This optimization focuses on a parameter set θ (including friction coefficients and control gains) to minimize the trajectory tracking error between open-loop

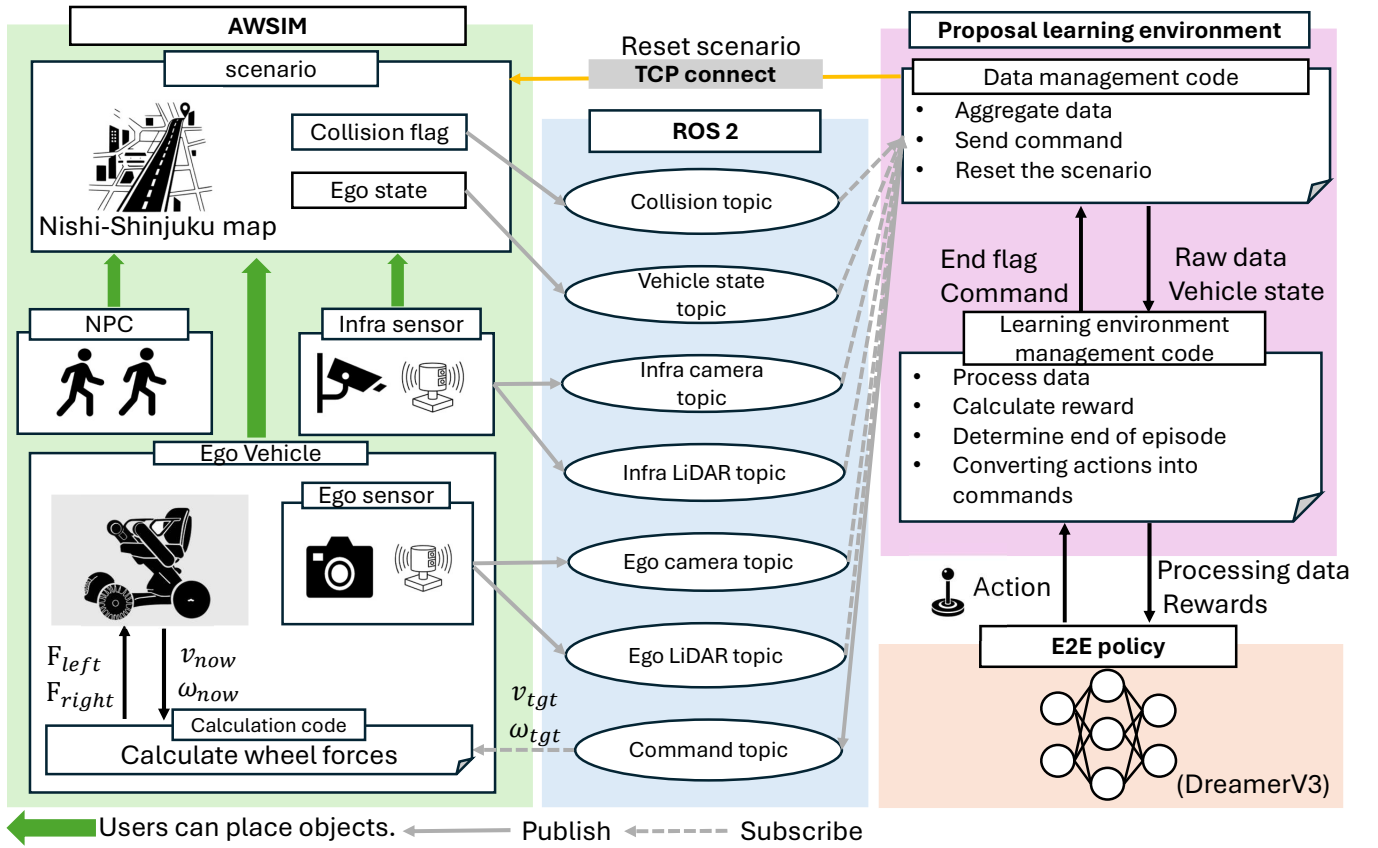


Fig. 2: Detailed System Model

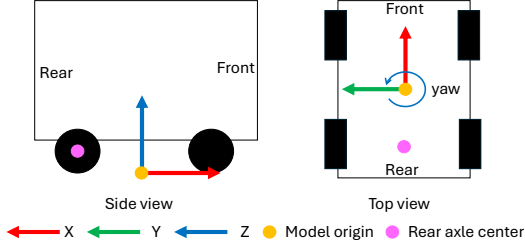


Fig. 3: Definition of the vehicle coordinate system and reference points.

executions on the physical vehicle and the simulation. The objective function $J(\theta)$ is the sum of the Root Mean Square Errors (RMSE) across a set of driving scenarios S :

$$J(\theta) = \sum_{k \in S} \sqrt{\frac{1}{T_k} \sum_{t=1}^{T_k} \|\mathbf{p}_{sim}^{(k)}(t, \theta) - \mathbf{p}_{real}^{(k)}(t)\|^2} \quad (1)$$

where T_k is the total steps in scenario k , and S comprises five predefined maneuvers (Straight, Turn, Circle, S-curve, and Figure-eight) designed to capture the longitudinal and non-linear lateral dynamics of the differential drive system. Here, $\mathbf{p}_{sim}$ represents the ground-truth 2D position of the rear axle center of the vehicle (Fig. 3), extracted from the global coordinates of the simulator. Correspondingly, $\mathbf{p}_{real}$ denotes the same rear axle center position of the physical vehicle, estimated via wheel odometry. For a kinematic comparison, both trajectories are aligned to a common origin at the start

TABLE II: Identified system parameters for the vehicle dynamics model

Symbol	Description (Unit)	Initial Values	Optimized Values*
Physics (Skidding & Slip)			
c_{rear}	Lateral damping coefficient (rear) [1/s]	1.000	34.79
c_{front}	Lateral damping coefficient (front) [1/s]	0.1000	0.4056
η	Forward slip multiplier [-]	1.000	0.8400
PID Control			
K_p	Proportional gain [1/s]	10.00	27.70
K_i	Integral gain [1/s ²]	0.0000	0.4025
K_d	Derivative gain [-]	0.0000	0.7384
I_{clamp}	Anti-windup limit [m]	0.5000	1.431
Resistance			
C_d	Air drag coefficient [-]	0.2000	1.132
μ_r	Rolling resistance coefficient [-]	0.002000	0.003300
A	Frontal area [m ²]	0.8000	1.062
CoM Offsets			
CoM_x	Longitudinal [m]	0.0000	-0.2654
CoM_y	Lateral [m]	0.0000	0.004400
CoM_z	Vertical [m]	0.0000	0.1215

*Optimized values were obtained using training data from four scenarios: Straight, Turn, Circle, and S-curve.

of each trial. The optimized parameter set θ was determined through 500 trials of Bayesian optimization (TPE [13]) conducted via Optuna [14]. The initial and optimized values for the target parameters are presented in Table II.

The identified parameters should be interpreted as simulator-level corrections rather than independently measured physical constants, with the dominant mismatch attributed to lateral slip and yaw-coupled motion.

C. E2E Learning and Deployment Architecture

The architecture enables DreamerV3 training and deployment to the vehicle. An environment wrapper converts raw sensor streams into model-ready tensors (e.g., 64×64 images). Furthermore, this wrapper bridges the action space by mapping the normalized continuous actions output by the DreamerV3 agent ($a_t = [a_v, a_\omega] \in [-1, 1]^2$) to standard `geometry_msgs/Twist` commands. These normalized values are linearly scaled to predefined velocity limits (v_{max} and ω_{max}) tailored to the specific experimental scenario. This transformation ensures that the generated commands remain within kinematically feasible and safe boundaries for the vehicle. Because both the AWSIM and the physical WHILL Model CR share the identical ROS 2 interface, the trained E2E policy is deployed directly to the real world without code modification.

A primary challenge in coupling asynchronous ROS 2 middleware with episodic reinforcement learning is synchronization. To prevent state mismatches during environment resets, a Hybrid Communication Protocol is implemented: high-bandwidth sensor data streams continuously via ROS 2, while state-management signals (e.g., episode resets) are transmitted via a synchronous TCP connection. To ensure the low-latency observation required for policy stability, the synchronization of asynchronous ROS 2 messages follows the specifications defined in Table III. Since the simulation physics and the inference process of the agent operate asynchronously, the subscriber queue size for high-bandwidth sensors is limited to a depth of 1. This drop-oldest policy ensures that the agent always retrieves the most recent frame available at the moment of inference, discarding any stale data accumulated due to processing jitter.

In the proposed environment, the decision step of the Markov Decision Process (MDP) is defined by the arrival of a new camera frame. Let Δt_{dec} denote the target inter-arrival time of these decision steps, configured to approximately 100 ms (10 Hz). At this moment, all other sensor values, such as odometry and collision flags, are sampled from the most recent buffered ROS 2 messages to construct the observation vector. To maintain $\Delta t_{dec} \approx 100$ ms across domains without modifying the core E2E policy, the temporal synchronization strategy is adopted. During simulation-based training, a timestamp-based blocking synchronization waits for the image timestamp to update, synchronizing the inference step with the simulator rendering speed to prevent state mismatches. Conversely, during real-world deployment, the system operates asynchronously using a fixed 10 Hz timer callback. At every timer event, the policy infers an action using the most recently buffered image frame, ensuring real-time responsiveness regardless of sensor or processing latency. Despite the operational difference between simulator-driven and wall-clock-driven approaches, both modes utilize 64×64 RGB image inputs and map to the same continuous velocity commands.

TABLE III: Specifications of ROS 2 topics and synchronization settings

Topic Name	Type	Hz	QoS	Stmp	Q
/sensing/camera/front/image_raw	sensor_msgs/Image	10	B.E.	Yes	1
/sensing/lidar/top/pointcloud_raw	sensor_msgs/PointCloud2	10	B.E.	No	1
/ground_truth/odom	nav_msgs/Odometry	20	B.E.	No	1
/collision_sensor	std_msgs/Bool	Ev.	Rel.	No	10
/whill/controller/cmd_vel	geometry_msgs/Twist	10	Rel.	No	1

Note: QoS indicates Reliability (B.E.: Best Effort, Rel.: Reliable). All topics use Drop Oldest policy. **Stmp**: Timestamp dependency. **Q**: Subscriber queue size. Ev.: Event-driven.

IV. EVALUATION

The evaluation addresses three dimensions: the reduction of the kinematic sim-to-real gap (RQ1), the reward curve of the world model-based policy for obstacle avoidance (RQ2), and direct sim-to-real deployment (RQ3).

A. Experimental Setup

Simulations and training utilize a workstation equipped with an AMD Ryzen Threadripper 7960X (24 cores), 256 GB of RAM, and an NVIDIA RTX 6000 Ada GPU. The software stack consists of Ubuntu 22.04.5 LTS, ROS 2 Humble, AWSIM v2.0.0, and Python 3.10 with the JAX and Optuna frameworks. For the sim-to-real transfer, the onboard computer of the physical WHILL Model CR operates on an identical Ubuntu and ROS 2 Humble environment.

B. Reduction of Kinematic Sim-to-Real Gap

To evaluate the fidelity of the E2E learning environment (RQ1), real-world odometry and control inputs were acquired across five scenarios: Straight, Turn, Circle, S-curve, and Figure-eight. By replaying the real-world control inputs in AWSIM (open-loop), the RMSE for trajectory, velocity, angular velocity, and yaw was calculated. The optimized trajectories align with the real-world data, as illustrated in Fig. 4, suppressing the outward drift seen in the initial model during complex maneuvers. This spatial accuracy is quantitatively supported by Table IV, which shows a reduction in trajectory RMSE across the evaluated scenarios. In the unseen Figure-eight test case, the optimization decreased the trajectory RMSE from 0.853 ± 0.002 m to 0.111 ± 0.002 m. This result confirms that the proposed system identification generalized the physical characteristics of the vehicle without overfitting. Furthermore, the temporal alignment of the yaw angle, illustrated in Fig. 5, highlights that the identified parameters compensate for the non-linear friction and lateral damping inherent to the differential drive system. While the initial model did not capture lateral dynamics, the optimized model matched the real-world angular responses. This improvement is validated by the yaw RMSE metrics in Table IV. For instance, the yaw error in the Turn scenario was reduced from 0.615 ± 0.008 rad to 0.064 ± 0.022 rad. While significant improvements were observed in trajectory and yaw metrics,

TABLE IV: Comparison of metrics across different scenarios and parameters. Values represent the mean $\pm$ SD of 10 independent trials. P -values are relative to the initial parameters

Scenario	Parameter	Trajectory RMSE [m]	Velocity RMSE [m/s]	Angular velocity RMSE [rad/s]	Yaw RMSE [rad]
Straight	Initial Values	0.166 ± 0.060	0.060 ± 0.004	0.027 ± 0.001	0.003 ± 0.000
	Optimized Values	0.058 ± 0.004 ($p < .001$)	0.067 ± 0.003 ($p < .001$)	0.028 ± 0.001 ($p = .276$)	0.004 ± 0.000 ($p < .001$)
Turn	Initial Values	0.406 ± 0.001	0.178 ± 0.000	0.068 ± 0.002	0.615 ± 0.008
	Optimized Values	0.002 ± 0.000 ($p < .001$)	0.178 ± 0.000 ($p = .272$)	0.043 ± 0.001 ($p < .001$)	0.064 ± 0.022 ($p = .002$)
Circle	Initial Values	0.846 ± 0.003	0.063 ± 0.001	0.076 ± 0.002	0.547 ± 0.005
	Optimized Values	0.037 ± 0.002 ($p < .001$)	0.037 ± 0.002 ($p < .001$)	0.056 ± 0.003 ($p < .001$)	0.055 ± 0.025 ($p < .001$)
S-curve	Initial Values	0.446 ± 0.002	0.044 ± 0.002	0.077 ± 0.002	0.093 ± 0.002
	Optimized Values	0.112 ± 0.003 ($p < .001$)	0.039 ± 0.002 ($p < .001$)	0.069 ± 0.002 ($p < .001$)	0.074 ± 0.002 ($p < .001$)
Figure-eight	Initial Values	0.853 ± 0.002	0.066 ± 0.001	0.077 ± 0.003	0.189 ± 0.004
	Optimized Values	0.111 ± 0.002 ($p < .001$)	0.049 ± 0.001 ($p < .001$)	0.066 ± 0.002 ($p < .001$)	0.118 ± 0.004 ($p < .001$)

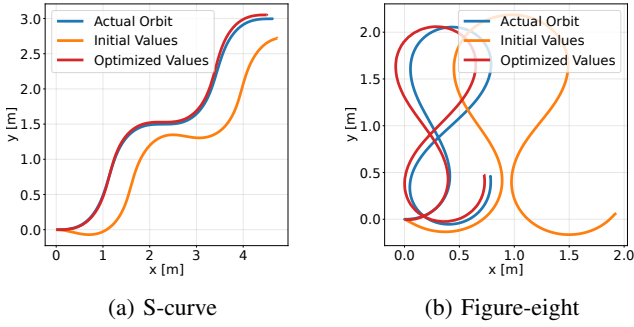


Fig. 4: Trajectories in S-curve and Figure-eight scenarios.

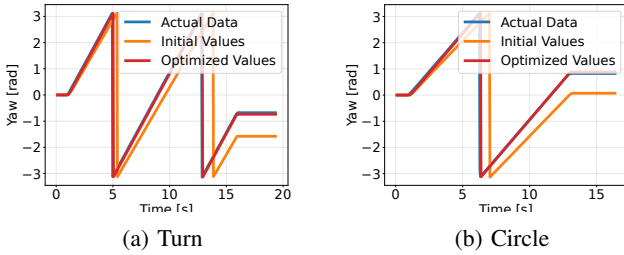


Fig. 5: Yaw angle in Turn and Circle scenarios.

the RMSE for velocity and angular velocity in Table IV shows relatively minor changes between the initial and optimized models, remaining consistently low. This result indicates that the foundational differential drive formulation in the initial model was already sufficient for tracking basic longitudinal and rotational commands. Consequently, the optimization process isolated and corrected the non-linear lateral dynamics, such as sideslip and friction, which were the primary sources of trajectory drift, without substantially degrading the core velocity tracking performance.

C. Efficacy of E2E Policy Training

To evaluate the training platform (RQ2), an obstacle avoidance scenario is designed (Fig. 6), where the agent navigates

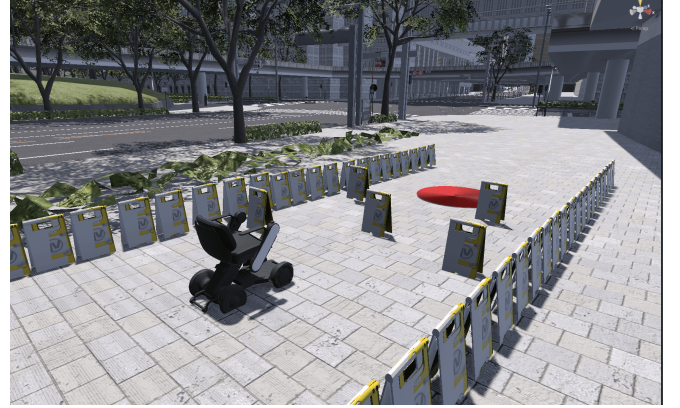


Fig. 6: A scenario with obstacles along the way to the goal.

to a target using only 64×64 RGB camera images as input. This evaluation focuses on whether the proposed AWSIM-ROS 2 environment can support the selected world model-based policy, rather than on a comprehensive algorithm benchmark. DreamerV3 is used because its recurrent latent world model is well matched to episodic visual navigation with sparse terminal events, enabling the agent to predict task progress and collision risk from image histories while planning over continuous velocity commands. For the simulated obstacle avoidance scenario, the predefined physical velocity limits are configured to $v_{max} = 1.0$ m/s and $\omega_{max} = 1.0$ rad/s. Accordingly, the normalized continuous actions output by the agent are linearly scaled to a linear velocity of $v \in [-1.0, 1.0]$ m/s and an angular velocity of $\omega \in [-1.0, 1.0]$ rad/s. The reward function is formulated to balance goal achievement, safety, and driving smoothness. To prevent the learning signal from being corrupted by odometry drift, all distance-based rewards are computed using the ground-truth coordinates provided by the AWSIM simulator rather than estimated wheel odometry. At each time step t , the total reward R_t is defined as $R_t = r_{goal} + r_{dist} + r_{collision} + r_{time} + r_{smooth}$. The goal reward r_{goal} is 100.0 if the rear axle center of the vehicle reaches within 1.0 m of

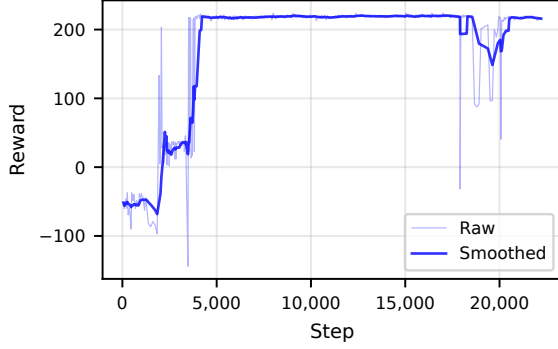


Fig. 7: Reward curve during training¹.

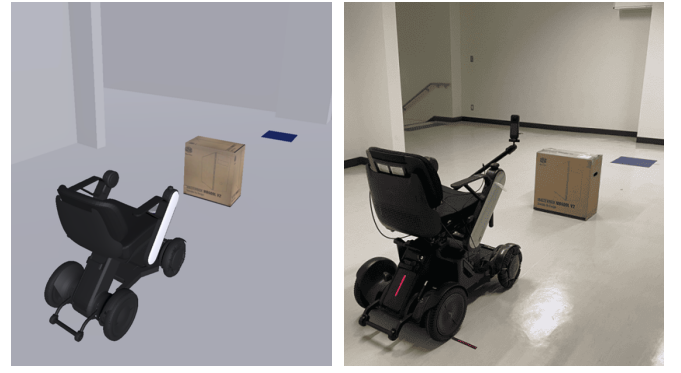
the goal center ($d_t \leq 1.0$), and 0.0 otherwise. The collision penalty $r_{collision}$ is -50.0 if physical contact is detected, and 0.0 otherwise. The distance reward $r_{dist} = 30.0(d_{t-1} - d_t)$ provides a dense progress reward proportional to the distance moved towards the goal. Given the maximum linear velocity $v_{max} = 1.0$ m/s and a control frequency of 10 Hz, r_{dist} is implicitly bounded within $[-3.0, 3.0]$ per step. A constant time penalty $r_{time} = -0.05$ is applied per step to encourage faster task completion. Finally, a smoothness penalty $r_{smooth} = -1.0|\omega_t - \omega_{t-1}|$ is applied during active movement to suppress abrupt changes in the commanded angular velocity ω_t (in rad/s).

An episode in the training and evaluation phases terminates under one of three conditions:

- **Success (Goal Arrival):** The episode concludes when the rear axle center of the vehicle reaches within a 1.0 m radius of the goal center.
- **Failure (Collision):** The episode terminates immediately as a failure if the vehicle makes physical contact with an obstacle or a wall.
- **Timeout:** To prevent infinite exploration loops, the episode is truncated if the agent fails to reach the goal within a maximum limit of 150 steps (corresponding to 15.0 seconds at 10 Hz).

The E2E policy exhibited learning progress, as shown by the reward curve in Fig. 7. Following an initial exploration phase with frequent collisions, the policy demonstrated consistent goal arrivals by step 4,000. By step 18,000, performance stabilized with an average reward of approximately 217 and an arrival time of 5 seconds. Although a temporary performance drop occurred at step 19,000 attributable to exploration dynamics, the agent recovered and achieved task completion by step 22,000. This performance recovery, coupled with the ability to learn obstacle avoidance from low-resolution inputs, validates the proposed environment as a practical training foundation for resource-constrained micromobility.

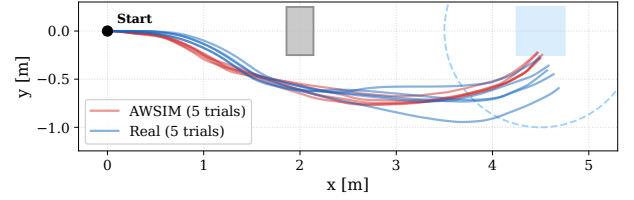
¹This curve shows the learning progress of the obstacle avoidance policy for RQ2 (22,000 steps). The 200,000-step training in Section IV-D is conducted for a separate policy designed for the sim-to-real corridor scenario (RQ3).



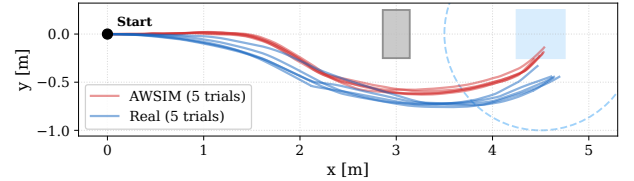
(a) AWSIM.

(b) Real-world.

Fig. 8: Visual comparison of the AWSIM and real-world.



(a) Obstacle at 2 m



(b) Obstacle at 3 m

Fig. 9: Comparison of AWSIM and real-world trajectories during sim-to-real obstacle avoidance. The blue dotted circle indicates the 1.0 m radius goal region.

D. Sim-to-Real Policy Transfer without Fine-Tuning

To evaluate the real-world applicability of the learned policy (RQ3), a direct sim-to-real transfer experiment without fine-tuning was conducted using the laboratory corridor environment introduced in Section III-A.

To establish a spatial reference, point cloud data of the physical corridor is acquired using the KISS-ICP SLAM algorithm [15] via LiDAR. The generated 3D mesh is used to manually model a lightweight, block-based environment using Unity cube assets (Fig. 8(a)). Within this environment, a 0.5×0.5 m blue square is placed 4.5 m ahead of the vehicle as the goal, alongside a cardboard box as an obstacle, matching the physical setup (Fig. 8(b)). The E2E driving policy is trained for 200,000 steps in this AWSIM environment. To ensure safety during physical navigation, the physical velocity limits are constrained to $v_{max} = 0.3$ m/s and $\omega_{max} = 0.6$ rad/s. The environment wrapper linearly scales the normalized actions from the trained policy to the velocity ranges of $v \in [0.0, 0.3]$ m/s and $\omega \in [-0.6, 0.6]$ rad/s. The training utilized the reward formulation defined in Section IV-C, excluding the smoothness penalty (r_{smooth}), resulting in $R_t = r_{goal} + r_{collision} + r_{dist} + r_{time}$. The

termination conditions remain identical, except the timeout is extended to 300 steps (30 s). Additionally, the evaluation trajectories in Fig. 9 depict 20.0-second trials.

To bridge visual and spatial gaps, domain randomization (DR) is used during training. Spatial randomization includes perturbing the initial pose of the vehicle (yaw angle between $[-15^\circ, 15^\circ]$, and lateral/longitudinal positions by ± 0.3 m) and the placement of the obstacle (longitudinal translation by ± 1.0 m, lateral translation by ± 0.5 m). To increase the robustness of the visual representations, data augmentation is applied to the camera inputs, randomizing brightness (0.8 to $1.3\times$), contrast (0.8 to $1.2\times$), hue (-0.1 to 0.1), and saturation (0.8 to $1.2\times$), alongside the addition of Gaussian noise (standard deviation 0 to 5). For the physical deployment, the policy weights obtained at 200,000 steps are directly transferred to the WHILL Model CR without fine-tuning. The evaluation focuses on comparing avoidance trajectories between the simulator and the real world. The cardboard obstacle is placed at two longitudinal distances (2 m and 3 m) ahead of the vehicle, with the goal fixed at 4.5 m. The blue dotted circle in Fig. 9 represents the 1.0 m radius goal range where the success reward triggers. The wide acceptance radius was designed to account for the limited field of view of the camera; a tight goal region would cause the visual target to enter the frontal blind spot of the vehicle immediately before arrival, destabilizing the final approach.

As observed in Fig. 9, the physical vehicle executed collision avoidance maneuvers in all real-world trials using only the live RGB camera feed. The trajectory data shows that the vehicle initiates evasive maneuvers at different longitudinal positions depending on the obstacle placement. This shift in avoidance timing indicates that the policy did not overfit to a single memorized steering pattern. Instead, the agent learned to recognize the spatial position of the obstacle from visual observations and generate avoidance commands. While deviations between the simulated and physical trajectories exist, these deviations can be attributed to inaccuracies in the manual initial positioning. Despite the initialization errors, the overall kinematic correlation validates the effectiveness of the system identification framework proposed in Section III. Moreover, the trajectory analysis reveals a wider lateral variance in the real vehicle compared to the simulation when navigating toward the final goal. While stemming partially from residual visual domain gaps, such as lighting and material differences, this variance highlights a characteristic of the learned policy. Instead of attempting to align with the center of the visual marker, the agent prioritizes safely clearing the obstacle to avoid collision penalties and entering the 1.0 m goal acceptance region. The observed behavior demonstrates that the combination of safety constraints and a wide goal reward formulation absorbs visual noise and initial state errors, facilitating consistent obstacle avoidance in the physical deployment.

E. Lessons Learned

This study yields three primary findings corresponding to the reduction of the kinematic sim-to-real gap, the reward

curve of the world model-based policy, and the direct physical deployment for micromobility vehicles.

RQ1: The optimization of physical parameters based on real-world driving data minimizes the sim-to-real gap in vehicle trajectories and yaw angles. The system identification framework generalized the physical characteristics without overfitting, reducing the trajectory RMSE from 0.853 m to 0.111 m in the unseen Figure-eight scenario. The optimization corrected non-linear lateral dynamics, such as sideslip and friction, while maintaining the core longitudinal velocity tracking performance.

RQ2: The E2E world model-based policy, using only RGB images, achieved curve for obstacle avoidance within the proposed environment. The policy demonstrated consistent goal arrivals by step 4,000 and reached stable performance by step 18,000 with an average reward of 217. This learning progression confirms that the proposed simulation setup provides a practical foundation for training micromobility autonomous driving systems.

RQ3: The vision-based policy trained exclusively in the simulator achieved collision-free obstacle avoidance on the physical vehicle without updating network weights. Trajectory data from the physical deployment indicates that the agent recognizes spatial obstacle positions from visual observations instead of executing memorized steering patterns. The combination of safety constraints and the goal acceptance region mitigates visual domain gaps and initial state errors during the direct sim-to-real transfer.

V. RELATED WORK

Simulation is crucial for safe autonomous driving, yet conventional car-centric platforms fail to capture the unique kinematics and shared pedestrian environments of micromobility vehicles. A review of existing simulation platforms, E2E learning for mobile robots, and sim-to-real transfer methods positions the contributions of this study.

A. Simulation Environments for Micromobility Vehicles

Simulators such as CARLA [16] and AirSim [17] primarily target automobiles. Recent micromobility-focused platforms, such as MetaUrban [18] and URBAN-SIM [19], prioritize procedural generation and large-scale learning over precise kinematic reproduction. Unlike these platforms, the proposed approach focuses on building an E2E learning environment for a specific region and vehicle model.

B. E2E Learning in Mobile Robots

E2E driving methods such as CarDreamer [20] and Follow-Me Wheelchair [21], as well as RL-based navigation algorithms (e.g., EPW-RL Navigation [22]), propose sophisticated control strategies. However, these methods rely on simplified kinematic models in simulation, which exacerbates the dynamic discrepancy during real-world deployment.

C. Sim-to-Real and System Identification

To bridge the sim-to-real gap, approaches such as Isaac-to-Real Mobile [23] utilize domain randomization [24]. In

TABLE V: Comparison of the proposed method with related research platforms and methods

Method (Reference)	SV	RM	PPT	R2	MF	ELR
CARLA [16]						✓
AirSim [17]						✓
MetaUrban [18]					✓	✓
URBAN-SIM [19]				✓	✓	✓
CarDreamer [20]						✓
Follow-Me Wheelchair [21]	✓			✓	✓	✓
EPW-RL Navigation [22]	✓				✓	✓
Isaac-to-Real Mobile [23]			✓	✓		✓
Proposed method	✓	✓	✓	✓	✓	✓

SV: Specific Vehicle, RM: Real-world Map, PPT: Physical Parameter Tuning, R2: ROS 2 Native Integration, MF: Micromobility Focus, ELR: E2E Learning Ready.

contrast, safely operating the WHILL Model CR demands precise system identification on the simulator side. As summarized in Table V, the proposed methodology explicitly incorporates physical parameter tuning within AWSIM, establishing a foundation for real-world deployment compared to existing studies.

VI. CONCLUSIONS

To bridge the gap between simulation and physical deployment, an E2E learning environment utilizing AWSIM and ROS 2 is developed for the kinematics of micromobility vehicles. The optimization of physical parameters via system identification reduced the kinematic sim-to-real gap, resulting in trajectory reproduction across diverse scenarios. Furthermore, the experimental results demonstrated that a world model-based agent (DreamerV3) trained solely within this simulated environment learned obstacle avoidance and achieved physical deployment without fine-tuning, enabling safe navigation in a physical corridor. Consequently, the proposed environment establishes a practical and validated foundation for developing and deploying E2E driving policies in real-world micromobility applications. Future work focuses on integrating multimodal sensors to ensure safe and efficient navigation in complex and dynamic environments.

ACKNOWLEDGMENT

This work was supported by JST FOREST Grant Number JPMJFR242G.

REFERENCES

- [1] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monroy, T. Ando, Y. Fujii, and T. Azumi, "Autoware on Board: Enabling Autonomous Vehicles with Embedded Systems," in *Proceedings of the 9th ACM/IEEE International Conference on Cyber-Physical Systems (ICCPs)*. IEEE, 2018, pp. 287–296.
- [2] Y. Tajima, S. Tsunoda, and T. Azumi, "ROS-lite2: Autonomous-driving Software Platform for Clustered Many-core Processor," in *Proceedings of the 36th IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 4578–4585.
- [3] H. Yu, W. Yang, J. Zhong, Z. Yang, S. Fan, P. Luo, and Z. Nie, "End-to-End Autonomous Driving Through V2X Cooperation," in *Proceedings of the 39th Association for the Advancement of Artificial Intelligence (AAAI) Conference on Artificial Intelligence*, vol. 39, no. 9, 2025, pp. 9598–9606.
- [4] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, "End-to-end Autonomous Driving: Challenges and Frontiers," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [5] J.-J. Hwang, R. Xu, H. Lin, W.-C. Hung, J. Ji, K. Choi, D. Huang, T. He, P. Covington, B. Sapp, Y. Zhou, J. Guo, D. Anguelov, and M. Tan, "EMMA: End-to-End Multimodal Model for Autonomous Driving," *arXiv preprint arXiv:2410.23262*, 2024.
- [6] T. Feng, W. Wang, and Y. Yang, "A Survey of World Models for Autonomous Driving," *arXiv preprint arXiv:2501.11260*, 2025.
- [7] A. Hu, L. Russell, H. Yeo, Z. Murez, G. Fedoseev, A. Kendall, J. Shotton, and G. Corrado, "GAIA-1: A Generative World Model for Autonomous Driving," *arXiv preprint arXiv:2309.17080*, 2023.
- [8] Y. Guan, H. Liao, Z. Li, J. Hu, R. Yuan, Y. Li, G. Zhang, and C. Xu, "World Models for Autonomous Driving: An Initial Survey," *IEEE Transactions on Intelligent Vehicles*, 2024.
- [9] P. Wu, A. Escontrela, D. Hafner, P. Abbeel, and K. Goldberg, "Day-Dreamer: World Models for Physical Robot Learning," in *Proceedings of the 7th Conference on Robot Learning (CoRL)*. PMLR, 2023, pp. 2226–2240.
- [10] S. Ito, C. Zhao, R. Okamura, and T. Azumi, "D-AWSIM: Distributed Autonomous Driving Simulator for Dynamic Map Generation Framework," in *Proceedings of the 28th Euromicro Conference on Digital System Design (DSD)*. IEEE, 2025, pp. 234–241.
- [11] Y. Maruyama, S. Kato, and T. Azumi, "Exploring the Performance of ROS2," in *Proceedings of the 13th International Conference on Embedded Software (EMSOFT)*, 2016, pp. 1–10.
- [12] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, "Mastering Diverse Domains through World Models," *arXiv preprint arXiv:2301.04104*, 2023.
- [13] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for Hyper-Parameter Optimization," *Advances in Neural Information Processing Systems*, vol. 24, 2011.
- [14] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2019, pp. 2623–2631.
- [15] I. Vizzo, T. Guadagnino, B. Mersch, L. Wiesmann, J. Behley, and C. Stachniss, "KISS-ICP: In Defense of Point-to-Point ICP – Simple, Accurate, and Robust Registration If Done the Right Way," *IEEE Robotics and Automation Letters*, vol. 8, no. 2, pp. 1029–1036, 2023.
- [16] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An Open Urban Driving Simulator," in *Proceedings of the 1st Conference on Robot Learning (CoRL)*. PMLR, 2017, pp. 1–16.
- [17] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles," in *Proceedings of the 11th International Conference on Field and Service Robotics (FSR)*. Springer, 2017, pp. 621–635.
- [18] W. Wu, H. He, J. He, Y. Wang, C. Duan, Z. Liu, Q. Li, and B. Zhou, "MetaUrban: An Embodied AI Simulation Platform for Urban Micromobility," *arXiv preprint arXiv:2407.08725*, 2024.
- [19] W. Wu, H. He, C. Zhang, J. He, S. Z. Zhao, R. Gong, Q. Li, and B. Zhou, "Towards Autonomous Micromobility through Scalable Urban Simulation," in *Proceedings of the 38th IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 27 553–27 563.
- [20] D. Gao, S. Cai, H. Zhou, H. Wang, I. Soltani, and J. Zhang, "CarDreamer: Open-Source Learning Platform for World Model Based Autonomous Driving," *IEEE Internet of Things Journal*, 2024.
- [21] S. Salimpour, I. Catalano, T. Westerlund, M. Falahi, and J. Peña-Queralt, "Follow-Me in Micro-Mobility with End-to-End Imitation Learning," *arXiv preprint arXiv:2511.05158*, 2025.
- [22] S. Chatzidimitriadis and K. Sirlantzis, "Deep Reinforcement Learning for Autonomous Navigation in Robotic Wheelchairs," in *Proceedings of the 3rd International Conference on Pattern Recognition and Artificial Intelligence (ICPRAI)*. Springer, 2022, pp. 271–282.
- [23] S. Salimpour, J. Peña-Queralt, D. Paez-Granados, J. Heikkonen, and T. Westerlund, "Sim-to-Real Transfer for Mobile Robots with Reinforcement Learning: from NVIDIA Isaac Sim to Gazebo and Real ROS 2 Robots," *arXiv preprint arXiv:2501.02902*, 2025.
- [24] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in *Proceedings of the 30th IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 23–30.